TP 2.5 : Exercices supplémentaires

Types avancés et sécurité PostgreSQL

Module : Bases de Données Avancées
Niveau : M1 **Responsable du TP :** M. Alachaher
Année universitaire : 2026/2027
Environnement : PostgreSQL / pgAdmin

Partie 1 — Bases de données objet-relationnelles

Rappel utile

Type composite : `(adresse).ville` ; tableau : `ANY`, `unnest()`, `array_append()` ; JSON simple : `preferences ->> 'langue'`.

Exercice 1 — Analyse croisée des données structurées

1. Afficher, pour chaque employé, son nom, son poste, sa ville de résidence et le nom de son département. Trier par département puis par nom.
2. Afficher les employés dont le tableau `competences` contient `SQL` et qui résident dans une ville différente de celle de leur département.
3. Pour chaque département, compter les employés qui préfèrent le télétravail (champ `teletravail` de `preferences`). Afficher aussi le nombre total d'employés du département.
4. Expliquer pourquoi l'accès à la ville d'une adresse composite et à un champ JSON utilise deux syntaxes différentes.

Exercice 2 — Tableaux et statistiques sur les compétences

1. Afficher une ligne par couple (employé, compétence), à l'aide de `unnest()`.
2. Compter le nombre d'employés possédant chaque compétence ; trier du plus fréquent au moins fréquent.
3. Ajouter la compétence `PostgreSQL` à l'employé 105 uniquement si elle n'est pas déjà présente dans son tableau. Vérifier le résultat.
4. Identifier les employés possédant au moins trois compétences et afficher leur nombre de compétences avec `cardinality()`.

Exercice 3 — Évolution des informations semi-structurées

1. Afficher les employés dont la langue préférée est `fr`, en extrayant cette information depuis la colonne JSON `preferences`.
2. Pour l'employé 106, ajouter ou remplacer dans `preferences` la clé `notification` avec la valeur booléenne `true`, en conservant les autres préférences. Utiliser le type `JSON`, sans `JSONB`.
3. Créer une colonne `telephones` de type `TEXT[]` dans `employe`. Affecter deux numéros fictifs à l'employé 107, puis afficher chaque numéro sur une ligne.
4. Réflexion : dans quel cas les téléphones devraient-ils plutôt être stockés dans une table relationnelle séparée ? Justifier.

Partie 2 — Gestion des utilisateurs, privilèges et sécurité

Consignes de sécurité

Effectuer les `CREATE ROLE`, `GRANT` et `REVOKE` avec le compte administrateur. Pour tester : `SET ROLE nom_role;` puis `RESET ROLE;`. Les tests de refus doivent être exécutés séparément pour éviter qu'une erreur n'interrompe une série de commandes. Utiliser un compte non superutilisateur autorisé à adopter les rôles. Vérifier l'absence de privilèges hérités via `PUBLIC` ou d'autres rôles.

Exercice 4 — Nouveau rôle : chargé de recrutement

1. Créer le rôle métier `charge_recrutement` sans connexion directe et lui accorder l'utilisation du schéma `public`.
2. Lui accorder la lecture des seules colonnes `id_employe`, `nom`, `prenom`, `poste` et `date_embauche` de `employe`, ainsi que la lecture de `departement`.
3. Activer ce rôle et tester successivement : consultation des noms et postes ; consultation des salaires ; modification d'un poste. Indiquer les opérations autorisées et refusées.
4. Revenir au rôle initial. Expliquer pourquoi `SELECT * FROM employe` peut échouer alors qu'une sélection de colonnes autorisées réussit.

Exercice 5 — Nouveau rôle : contrôleur des projets

1. Créer `controleur_projets` (`NOLOGIN`) et lui accorder `USAGE` sur le schéma `public`.
2. Autoriser la lecture de `projet` et `affectation`, mais uniquement la modification de la colonne `statut` de `projet`.
3. Sous ce rôle, tester : lecture des projets ; modification du statut du projet 201 ; modification de son budget ; suppression d'une affectation. Relever les messages obtenus.
4. Retirer le droit de modifier `statut`, puis refaire le test de modification. Quel principe de sécurité est illustré ?

Exercice 6 — Audit des droits et restriction progressive

1. Créer `assistant_marketing` (`NOLOGIN`) et autoriser uniquement la lecture des colonnes `id_projet`, `nom` et `statut` de `projet`.
2. Tester sous ce rôle la consultation des colonnes autorisées, puis la lecture de `budget` et de la table `employe`. Expliquer les refus.
3. Depuis le compte administrateur, consulter `information_schema.role_table_grants` et `information_schem` afin d'observer les droits accordés. Comparer les résultats avec les tests précédents.
4. Accorder temporairement au rôle la lecture de `budget`, vérifier son effet, puis révoquer ce privilège. Expliquer l'intérêt de la révocation et du principe du moindre privilège.

Bilan

Vous devez être capables de combiner types composites, tableaux et JSON simple ; puis de créer des rôles métier, accorder des privilèges par table ou colonne, vérifier les restrictions et révoquer les droits inutiles.